demo 中 `protocol_lib` 与 `flutter_bluetooth` 使用流程说明

1. 文档目的

本文基于 `demo` 的实际源码，梳理以下两个本地依赖库在 demo 中的接入方式、职责边界、调用流程与典型用法：

- `protocol_lib`
- `flutter_bluetooth`

对应 `pubspec.yaml` 配置见 pubspec.yaml：

```yaml
protocol_lib:
  path: ../../protocol_lib/
protocol_alexa:
  path: ../../protocol_alexa/
flutter_bluetooth:
  path: ../../../bluetooth_sdk/
```

本文重点不是解释 Flutter 页面本身，而是说明：

1. 这两个库在 demo 中分别负责什么
2. 它们如何被初始化
3. 它们如何通过桥接形成完整的数据闭环
4. 页面层应该如何调用它们的能力

---

2. 两个库的职责划分

2.1 `flutter_bluetooth` 的职责

`flutter_bluetooth` 是蓝牙传输层，负责：

- 注册蓝牙 SDK
- 获取蓝牙状态
- 扫描设备
- 连接 / 断开设备
- BLE / SPP 数据写入
- 接收设备上报原始字节流

- 上报蓝牙状态、连接状态、写入状态、扫描结果

入口导出见 ido_bluetooth.dart。

全局单例入口：

```
final bluetoothManager = IDOBluetoothManager();
```

定义见 ido_bluetooth_manager.dart。

2.2 `protocol_lib` 的职责

`protocol_lib` 是协议业务层，负责：

- 注册协议库
- 维护当前接入设备的协议上下文
- 处理设备连接/断开后的协议状态切换
- 发送协议指令
- 解析设备返回数据
- 提供绑定、设备信息、功能表、同步、文件传输、消息图标、日志等上层能力

入口导出见 protocol_lib.dart。

全局单例入口：

```
final libManager = IDOProtocolLibManager();
```

定义见 ido_protocol_lib.dart。

2.3 两者的边界

这两个库不是并列互相替代，而是上下层关系：

- `flutter_bluetooth` 负责“把字节送到设备、把字节从设备拿回来”
- `protocol_lib` 负责“决定发什么协议包、如何解析返回包、如何组织业务能力”

页面层原则上不应自己拼装 BLE 原始字节，也不应直接解析设备协议数据。页面应：

- 用 `flutter_bluetooth` 做扫描和连接
- 用 `protocol_lib` 做绑定、查询、设置、同步、文件传输

---

3. demo 的核心架构

demo 的核心逻辑集中在 main.dart。

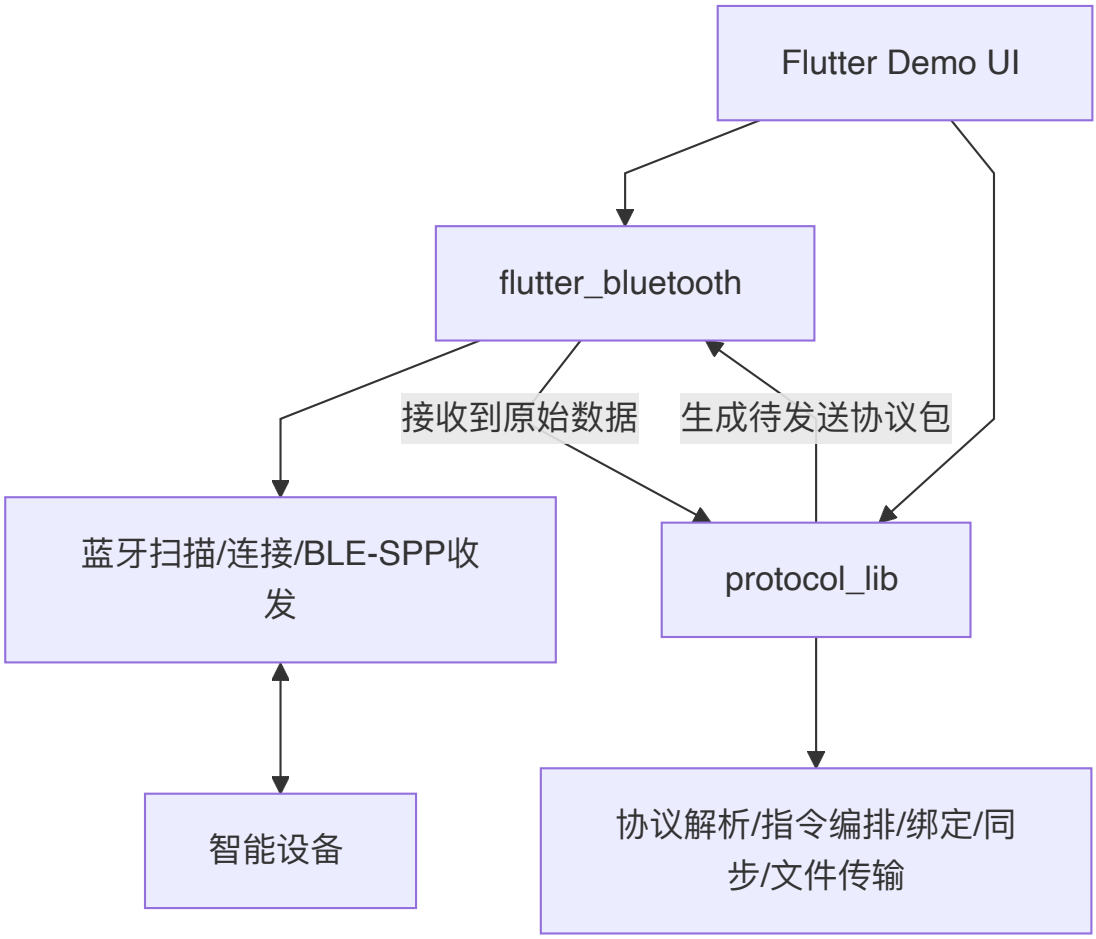
应用启动后，按以下顺序执行：

1. `runApp(const MyApp())`
2. `registerProtocolSDK()`
3. `registerBluetoothSDK()`
4. `registerProtocolAlexa()`
5. `bridgeConnect()`

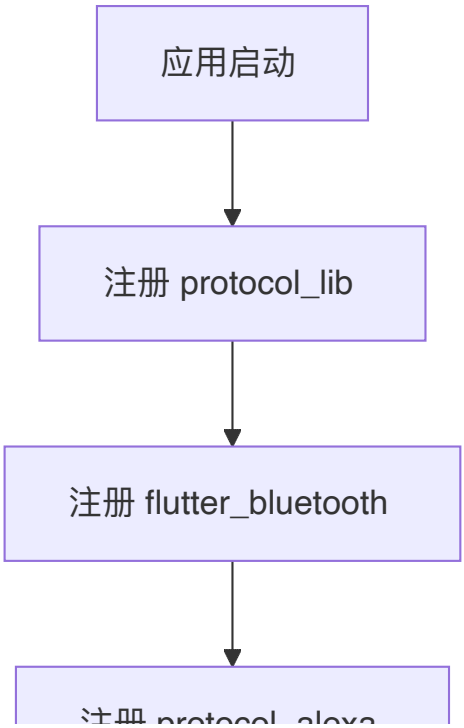
对应代码：

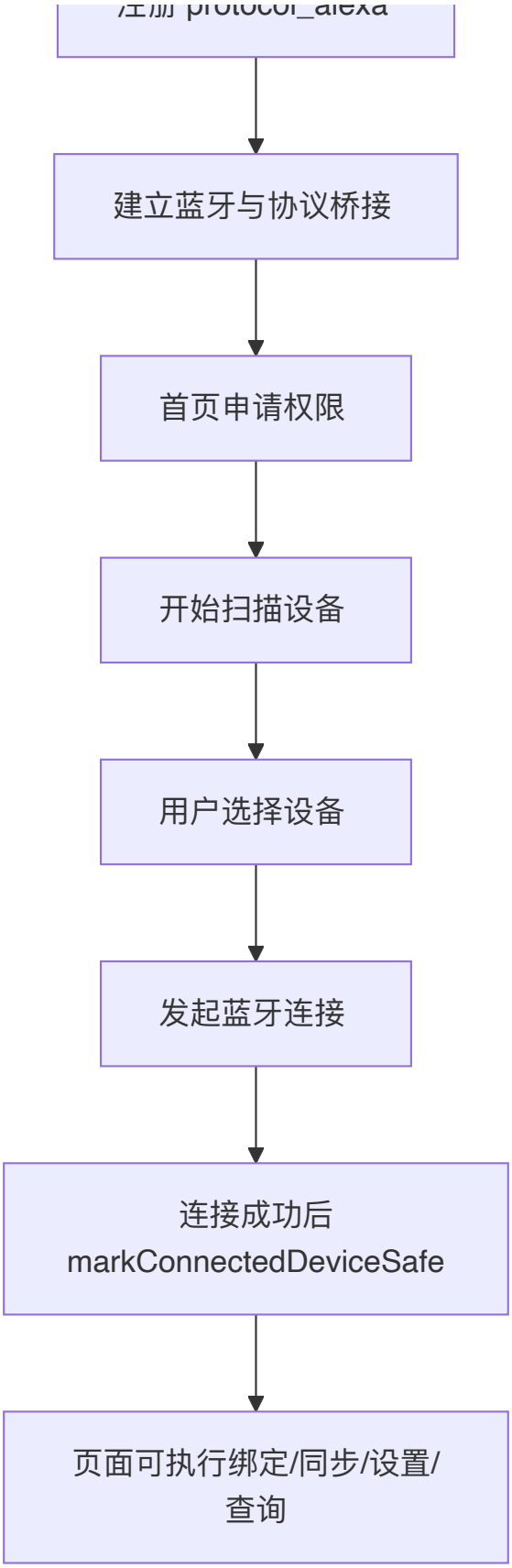
```
void main() async {
  runApp(const MyApp());

  await registerProtocolSDK();
  await registerBluetoothSDK();
  await registerProtocolAlexa();
  await bridgeConnect();
}
```

3.1 总体架构图


Flutter Demo UI
flutter_bluetooth
接收到原始数据
生成待发送协议包
蓝牙扫描/连接/BLE-SPP收发
protocol_lib
智能设备
协议解析/指令编排/绑定/同步/文件传输

3.2 启动阶段流程图


应用启动
注册 protocol_lib
注册 flutter_bluetooth


注册 protocol_alexa
建立蓝牙与协议桥接
首页申请权限
开始扫描设备
用户选择设备
发起蓝牙连接
连接成功后
markConnectedDeviceSafe
页面可执行绑定/同步/设置/
查询

> **Important**
> **`markConnectedDeviceSafe(...)` 和 `markDisconnectedDevice(...)` 是 `protocol_lib` 接入设备生命周期的关键入口，必须成对出现。**
>
> 必须固定遵守这条规则：
>
> - **蓝牙连接成功后，调用 `markConnectedDeviceSafe(...)`**
> - **蓝牙连接断开、主动取消连接、系统蓝牙关闭后，调用 `markDisconnectedDevice(...)`**
>
> 如果只调前者，不调后者，`protocol_lib` 可能一直保留错误的在线状态。
> 如果只调后者，不调前者，`protocol_lib` 则无法建立当前设备的协议上下文。

---

4. 启动与初始化流程

4.1 `protocol_lib` 初始化

demo 在 `registerProtocolSDK()` 中初始化协议库：

```
await IDOProtocolLibManager.register(outputToConsole: true);
libManager.messageIcon.registerListenUpdate();
libManager.listenDeviceNotification((event) {
  print('listenDeviceNotification: ${event.toMap().toString()}');
});
```

源码位置：

- main.dart:42
- ido_protocol_lib.dart

4.1.1 `register(...)` 的作用

`IDOProtocolLibManager.register(...)` 是协议层使用前的必要入口。它会完成：

- 初始化协议日志系统
- 初始化本地 KV 存储
- 初始化 C 库 / 协议运行环境

常用参数：

- `outputToConsole`
  - 是否把 SDK 日志输出到控制台
- `outputToConsoleClib`
  - 是否把 C 库日志输出到控制台
- `isReleaseClib`

    - 是否让 C 库按 release 模式运行

demo 中只打开了 `outputToConsole: true`。

4.1.2 初始化后立刻做的事情

demo 在协议库注册完成后，还做了两件事：

1. `libManager.messageIcon.registerListenUpdate()`
2. `libManager.listenDeviceNotification(...)`

说明：

- 第一项用于 iOS 消息图标更新监听
- 第二项用于监听设备主动上报的通知 / 控制事件

这类监听不一定是每个项目都必须照搬，但 demo 里作为协议层初始化的一部分已经接入。

4.2 `flutter_bluetooth` 初始化

demo 在 `registerBluetoothSDK()` 中初始化蓝牙库：

```
await bluetoothManager.register();
final version = bluetoothManager.getSdkVersion();
```

源码位置：

- main.dart:58
- ido_bluetooth_manager.dart

4.2.1 `register(...)` 的作用

蓝牙库注册时会做这些事：

- 初始化日志
- 注册平台通道
- iOS 启动心跳机制
- 主动获取蓝牙状态
- 监听蓝牙状态
- 监听设备连接状态
- 监听接收数据
- 监听扫描结果

也就是说，`bluetoothManager.register()` 不只是“简单初始化”，它已经开始接管蓝牙状态流。

4.2.2 `getSdkVersion()`

demo 获取了蓝牙 SDK 版本号，但没有继续显示在 UI 中。这个调用主要用于：

- 调试
- 日志上报
- 快速确认 SDK 集成版本

4.3 `protocol_alexa` 初始化

虽然本文重点是前两个库，但 demo 同时注册了 Alexa：

```
await IDOProtocolAlexa.register(clientId: clientId);
Connectivity().onConnectivityChanged.listen((event) {
  IDOProtocolAlexa.onNetworkChanged(hasNetwork: event != ConnectivityResult.none);
});
```

它与本文主线关系不大，但这里能看出一个事实：

- `protocol_lib` 和 `flutter_bluetooth` 是设备链路核心
- `protocol_alexa` 是建立在设备能力之上的业务扩展模块

---

5. 蓝牙层与协议层的桥接

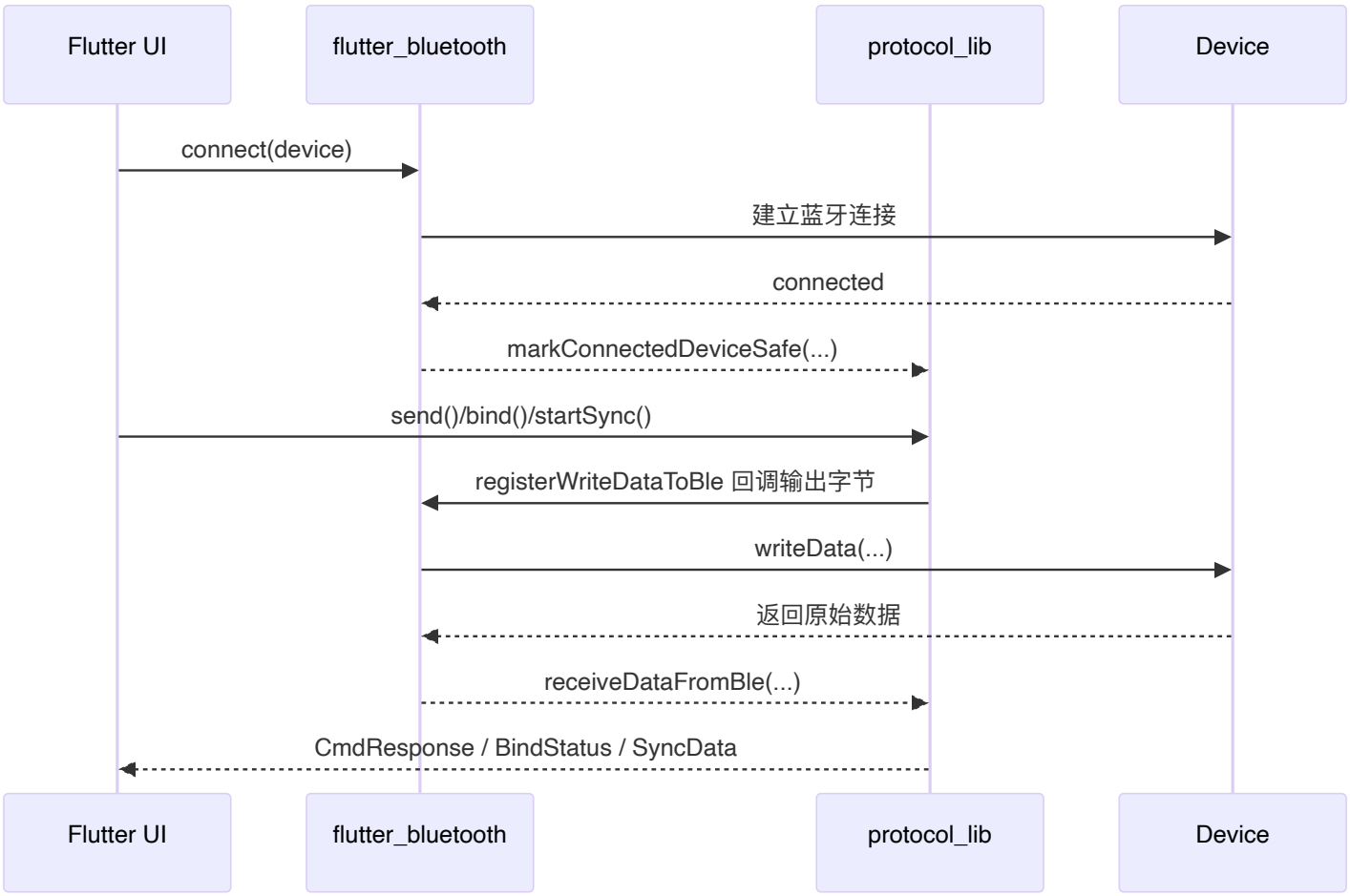
这是整个 demo 最关键的部分。代码在 main.dart 的 `bridgeConnect()`。

如果没有这层桥接：

- `flutter_bluetooth` 只能连设备、发字节、收字节
- `protocol_lib` 无法知道设备什么时候连接，也拿不到蓝牙返回的数据

桥接后，两个库才组成完整闭环。

5.1 桥接关系总览



5.2 把蓝牙收到的数据交给协议层

demo 写法：

```
bluetoothManager.receiveData().listen((event) {
  if (event.data != null) {
    libManager.receiveDataFromBle(
      event.data!,
      event.macAddress,
      event.spp ?? false ? 1 : 0,
    );
  }
});
```

源码位置：[main.dart:83]

说明：

- `receiveData()` 是蓝牙库的原始收包入口
- `receiveDataFromBle(...)` 是协议库的总收包入口

参数含义：

- `event.data!`
  - 设备返回的原始字节数据
- `event.macAddress`
  - 当前包对应设备地址
- `event.spp ?? false ? 1 : 0`
  - 数据通道类型
  - `0` 表示 BLE
  - `1` 表示 SPP

5.2.1 这一段为什么关键

只有调用了 `libManager.receiveDataFromBle(...)`，协议库才能：

- 解析响应包
- 唤醒对应命令的回调
- 更新设备信息、功能表、同步状态
- 触发 `listenStatusNotification(...)`
- 触发绑定 / 同步 / 文件传输 / 数据交换等上层模块回调

如果这一段缺失，页面会表现成：

- 连接成功
- 发送看似成功
- 但协议能力全部没有回调

5.3 把协议层要发送的数据交给蓝牙层

demo 写法：

```
IDOBluetoothWriteType rs = IDOBluetoothWriteType.withoutResponse;
libManager.registerWriteDataToBle((event) async {
  rs = await bluetoothManager.writeData(event.data, type: event.type);
  if (rs == IDOBluetoothWriteType.withoutResponse && Platform.isIOS) {
    libManager.writeDataComplete();
  }
});
```

源码位置：[main.dart:107]

说明：

- `registerWriteDataToBle(...)` 由协议库注册一个“实际发包函数”
- 协议库内部一旦有命令需要发送，就通过这个回调把 `CmdRequest` 抛给业务层
- demo 再调用 `bluetoothManager.writeData(...)` 真正发给设备

参数含义：

- `event.data`
  - 协议库已经拼装好的待发送字节流
- `event.type`
  - 写入类型
  - `0` 是 BLE
  - `1` 是 SPP

5.3.1 为什么页面层不应自己拼包

因为 `protocol_lib` 已经封装了：

- 指令队列
- 请求体编码
- 优先级
- 状态管理
- 超时与任务协调

页面层只需要负责把“协议库给出的字节”发出去，不应自己绕过 `protocol_lib` 直接构造包体。

5.4 写入完成后的确认回调

demo 处理了两种情况：

5.4.1 iOS 无响应写

```
if (rs == IDOBluetoothWriteType.withoutResponse && Platform.isIOS) {
  libManager.writeDataComplete();
}
```

原因：

- iOS 无响应写不会像有响应写那样再返回明确完成事件
- 所以在 `writeData(...)` 返回后，直接通知协议库“发送完成”

5.4.2 Android 或有响应写

```
bluetoothManager.writeState().listen((event) {
  if (event.state ?? false) {
    if (Platform.isAndroid || event.type == IDOBluetoothWriteType.withResponse) {
      libManager.writeDataComplete();
    }
  }
});
```

源码位置：main.dart:116

作用：

- 当蓝牙库确认一次写入完成后，再通知协议库继续调度命令流程

5.4.3 如果不调用 `writeDataComplete()`

协议库可能认为当前命令还没有完成底层发送，从而导致：

- 队列阻塞
- 后续命令不继续下发
- 看起来像“只发送第一包，后面都卡住”

---

6. 蓝牙扫描与设备选择流程

蓝牙扫描的页面逻辑在：

- home_page.dart
- home_content.dart

6.1 权限与蓝牙开关检查

点击首页右下角按钮后调用 `requestPermission()`。

demo 在 Android 下做了这些检查：

1. Android 29 及以上检查定位权限
2. 检查蓝牙开关是否已打开
3. 申请 `Permission.bluetoothScan`
4. 申请 `Permission.bluetoothConnect`
5. 调用 `bluetoothManager.startScan()`

源码位置：home_page.dart:39

6.1.1 为什么要先检查定位权限

这是 Android 蓝牙扫描常见前置条件。demo 中对 Android 29+ 做了显式判断：

- 如果没有定位权限，就弹出错误提示
- 不继续扫描

6.2 开始扫描

demo 直接调用：

```
bluetoothManager.startScan();
```

该方法定义在 ido_bluetooth_manager.dart。

从蓝牙库实现上看，`startScan([String? macAddress])` 支持：

- 不指定参数时扫描全部设备
- 指定 `macAddress` 时扫描指定设备

扫描前蓝牙库内部还会：

- 检查蓝牙状态
- 停止已有扫描
- 清理部分扫描缓存
- 启动原生侧扫描

6.3 获取扫描结果

首页内容组件在 `initState()` 中监听：

```
bluetoothManager.scanResult().listen((event) {
  setState(() {
    devices = event;
  });
});
```

源码位置：home_content.dart:19

这里返回的是：

```
Stream<List<IDOBluetoothDeviceModel>>
```

页面把每个设备展示为按钮，显示：

- `device.name`
- `device.macAddress`

6.4 选择设备

点击设备后，调用：

```
Navigator.push(context, MaterialPageRoute(builder: (context)=>FunctionIndex(device)));
```

源码位置：[home_content.dart:48]

这里有个很重要的设计点：

- 页面传递的是扫描结果返回的 `IDOBluetoothDeviceModel`
- 后续连接时直接用这个对象调用 `bluetoothManager.connect(device)`

这很重要，因为扫描返回对象里通常已经包含：

- `macAddress`
- `uuid`
- `deviceId`
- `platform`
- `isOta`
- `isTlwOta`

尤其是 iOS 的 `uuid`，后面协议层接入设备时会用到。

---

7. 连接设备后的处理流程

连接逻辑主要在 [function_index.dart] 与 [main.dart]。

> **Important**
> **本章最关键的不是“连接按钮怎么写”，而是：**
>
> - **连接成功后要调用 `markConnectedDeviceSafe(...)`**
> - **连接结束后要调用 `markDisconnectedDevice(...)`**
>
> **这两个方法必须成对出现。**

7.1 发起连接

功能页里，点击“连接”按钮时：

```
bluetoothManager.connect(device);
```

源码位置：[function_index.dart:311]

断开时：

```
bluetoothManager.cancelConnect();
```

源码位置：[function_index.dart:315]

7.2 监听蓝牙连接状态

demo 中有两处监听 `bluetoothManager.deviceState()`：

- 全局桥接处，在 `main.dart`
- 功能页中，在 `function_index.dart`

这两处职责略有不同：

- `main.dart`
  - 负责协议层接入 / 断开同步
- `function_index.dart`
  - 负责页面状态刷新，比如是否允许“连接 / 绑定 / 同步 / Alexa”按钮可点

7.3 蓝牙连接成功后，通知协议层设备已接入

> **Important**
> **`markConnectedDeviceSafe(...)` 不是普通通知，而是 `protocol_lib` 正式进入“设备在线协议态”的关键动作。**
>
> **它必须和 `markDisconnectedDevice(...)` 配对维护。**

桥接代码：

```
if ((value.state == IDOBluetoothDeviceStateType.connected &&
    (value.macAddress != null && value.macAddress!.isNotEmpty))) {
  final isTlwOta = bluetoothManager.currentDevice?.isTlwOta ?? false;
  final isOta = bluetoothManager.currentDevice?.isOta ?? false;
  final otaType = isTlwOta
      ? IDOOtaType.telink
      : isOta
          ? IDOOtaType.nordic
          : IDOOtaType.none;

  final devicenName = bluetoothManager.currentDevice?.name;
  var uniqueId = value.macAddress!;

  if (Platform.isIOS && bluetoothManager.currentDevice?.uuid != null) {
    uniqueId = bluetoothManager.currentDevice!.uuid!;
  }
```

```
  await libManager.markConnectedDeviceSafe(
      uniqueId: uniqueId,
      otaType: otaType,
      isBinded: false,
      deviceName: devicenName);
}
```

源码位置：[main.dart:127]

7.3.1 `markConnectedDeviceSafe(...)` 的意义

这个方法是协议层“进入在线状态”的关键。

> **Important**
> **可以直接把 `markConnectedDeviceSafe(...)` 理解为：告诉 `protocol_lib`，当前设备已经正式上线，可以建立协议上下文。**

调用后，协议库才知道：

- 当前连的是哪台设备
- 当前设备唯一标识是什么
- 当前设备是否为 OTA 模式
- 当前设备名称是什么
- 当前设备是否已绑定

如果不调用，虽然蓝牙已经连上，但 `protocol_lib` 仍可能认为当前没有接入设备。

这里要特别注意：

- **调用 `bluetoothManager.connect(...)` 不等于 `protocol_lib` 已经连上设备**
- **只有调用 `markConnectedDeviceSafe(...)` 后，`protocol_lib` 才真正进入可用状态**

7.3.2 `uniqueId` 的取值策略

源码注释已经说明：

- Android：传当前连接设备的 `macAddress`
- iOS：优先传 `uuid`，也可传 `macAddress` 或 `uuid`

demo 的实际处理是：

- 默认用 `macAddress`
- 如果是 iOS 且有 `uuid`，则改用 `uuid`

7.3.3 `otaType` 的来源

demo 从 `bluetoothManager.currentDevice` 上读取：

- `isTlwOta`
- `isOta`

转换为协议库需要的 `IDOOtaType`：

- `telink`
- `nordic`
- `none`

7.3.4 `isBinded` 在 demo 里的处理

在 `main.dart` 中，demo 直接写死传了 `false`。

而在 function_index.dart 中，页面级监听又会从本地缓存恢复绑定状态：

```
final isBinded = await libManager.cache.loadBindStatus(macAddress: event.macAddress!);
await libManager.markConnectedDeviceSafe(
  uniqueId: uniqueId,
  otaType: otaType,
  isBinded: isBinded,
);
```

这说明在正式项目里，更合理的做法是：

- 从本地缓存或业务系统恢复设备绑定状态
- 再把这个状态传给协议层

7.4 设备断开后，通知协议层已离线

> **Important**
> **`markDisconnectedDevice(...)` 不是可选清理，而是 `protocol_lib` 正式退出“设备在线协议态”的关键动作。**
>
> **它必须和 `markConnectedDeviceSafe(...)` 成对出现。**

demo 中，当蓝牙状态变为 `disconnected` 时，会调用：

```
await libManager.markDisconnectedDevice(
    macAddress: value.macAddress, uuid: value.uuid);
```

源码位置：main.dart:160

同时，在手机蓝牙关闭时，也会调用：

```
await libManager.markDisconnectedDevice(
    macAddress: macAddress, uuid: uuid);
```

源码位置：[main.dart:166]

这一步用于：

- 清理协议在线状态
- 阻止页面继续调用在线能力
- 避免协议库误认为当前设备仍可通信

这里同样要特别注意：

- **蓝牙库断开了，不代表 `protocol_lib` 一定已经退出在线态**
- **只有调用 `markDisconnectedDevice(...)` 后，`protocol_lib` 才完成协议下线**

7.4.1 成对使用规则

> **Important**
> **推荐把下面两条写进项目接入规范：**
>
> 1. **任意一次有效连接建立后，必须调用一次 `markConnectedDeviceSafe(...)`**
> 2. **任意一次连接生命周期结束后，必须调用一次 `markDisconnectedDevice(...)`**
>
> **最稳妥的做法是统一放在蓝牙状态监听里，而不是散落在页面按钮逻辑中。**

---

8. 页面层如何使用 `flutter_bluetooth`

这一部分只总结 demo 里已经实际使用到的 `flutter_bluetooth` 典型入口。

8.1 注册

```
await bluetoothManager.register();
```

8.2 获取版本

```
final version = bluetoothManager.getSdkVersion();
```

8.3 获取蓝牙开关状态

```
IDOBluetoothStateType bleState = await bluetoothManager.getBluetoothState();
```

8.4 开始扫描

```
bluetoothManager.startScan();
```

8.5 停止扫描

```
bluetoothManager.stopScan();
```

首页左上角按钮就是直接停止扫描，见 home_page.dart:23。

8.6 监听扫描结果

```
bluetoothManager.scanResult().listen((event) { ... });
```

8.7 发起连接

```
bluetoothManager.connect(device);
```

8.8 取消连接

```
bluetoothManager.cancelConnect();
```

8.9 监听设备连接状态

```
bluetoothManager.deviceState().listen((event) { ... });
```

8.10 监听蓝牙开关状态

```
bluetoothManager.bluetoothState().listen((event) { ... });
```

8.11 写数据

```
await bluetoothManager.writeData(event.data, type: event.type);
```

8.12 监听写入状态

```
bluetoothManager.writeState().listen((event) { ... });
```

8.13 监听接收数据

```
bluetoothManager.receiveData().listen((event) { ... });
```

---

9. 页面层如何使用 `protocol_lib`

9.1 注册

```
await IDOProtocolLibManager.register(outputToConsole: true);
```

9.2 标记设备已接入

```
await libManager.markConnectedDeviceSafe(
  uniqueId: uniqueId,
  otaType: otaType,
  isBinded: isBinded,
  deviceName: devicenName,
);
```

> **Important**
> 这是 `protocol_lib` 的关键接入动作。
> 不要只调用这一个方法，而遗漏对应的 `markDisconnectedDevice(...)`。

9.3 标记设备已断开

```
await libManager.markDisconnectedDevice(
  macAddress: value.macAddress,
  uuid: value.uuid,
);
```

> **Important**
> 这是 `protocol_lib` 的关键退出动作。
> 必须和 `markConnectedDeviceSafe(...)` 配对维护。

9.4 监听协议状态

```
libManager.listenStatusNotification((status) async { ... });
```

demo 中关心的状态有：

- `protocolConnectCompleted`

- `fastSyncCompleted`
- `deviceInfoUpdateCompleted`
- `unbindOnBindStateError`
- `fastSyncFailed`

9.5 监听设备主动通知

```
libManager.listenDeviceNotification((event) { ... });
```

9.6 注册底层发包函数

```
libManager.registerWriteDataToBle((event) async { ... });
```

9.7 把蓝牙收包传入协议层

```
libManager.receiveDataFromBle(data, macAddress, type);
```

9.8 通用指令发送

demo 中大量“获取 / 设置”页面都使用：

```
libManager.send(
  evt: e.cmd,
  json: jsonEncode(e.json ?? {}),
).listen((res) { ... });
```

这说明在业务页面里，最通用的调用方式就是：

1. 选择一个 `CmdEvtType`
2. 组织 `json`
3. 监听 `CmdResponse`

常见使用位置：

- function_get.dart
- heartRateMode_set.dart
- longSitRemind_set.dart

9.9 绑定

demo 的绑定调用方式：

```
libManager.deviceBind
    .startBind(
      osVersion: 15,
      deviceInfo: (d) {},
      functionTable: (f) {},
    )
    .listen((event) { ... });
```

源码位置：[function_index.dart:227]

`BindStatus` 可能返回：

- `failed`
- `successful`
- `binded`
- `needAuth`
- `refusedBind`
- `wrongDevice`
- `authCodeCheckFailed`
- `canceled`
- `failedOnGetFunctionTable`
- `failedOnGetDeviceInfo`
- `timeout`
- `agreeDeleteDeviceData`
- `denyDeleteDeviceData`
- `timeoutOnNewAccount`
- `needConfirmByApp`
- `accountNotMatch`

定义见 [device_bind_impl.dart]。

9.9.1 绑定内部大致流程

从源码看，`startBind(...)` 内部会做：

1. 检查 `libManager.isConnected`
2. 清理旧指令队列
3. 停止快速配置
4. 获取设备信息

5. 回调 `deviceInfo(...)`
6. 获取功能表
7. 回调 `functionTable(...)`
8. 根据设备回应输出对应 `BindStatus`

9.9.2 解绑

demo 的解绑调用方式：

```
libManager.deviceBind.unbind(macAddress: libManager.macAddress).then((res) {
  ...
});
```

源码位置：[function_index.dart:282]

解绑既支持：

- 当前在线设备解绑
- 离线设备解绑清理缓存

这是 `protocol_lib` 相对纯 BLE 库更高层的一个典型体现。

9.10 数据同步

同步页面在 [function_sync.dart]。

开始同步：

```
libManager.syncData.startSync(
  funcProgress: (progress) {
    debugPrint("progress: $progress");
  },
  funcData: (type, jsonStr, errorCode) async {
    ...
  },
  funcCompleted: (errorCode) {
    ...
  },
);
```

停止同步：

```
libManager.syncData.stopSync();
```

9.10.1 同步回调含义

- `funcProgress`
  - 整体同步进度
- `funcData`
  - 每种数据类型的同步结果
- `funcCompleted`
  - 所有同步结束

9.10.2 支持的数据类型

从 ido_sync_data.dart 可见，支持的同步数据包括：

- 步数
- 心率
- 睡眠
- 血压
- 血氧
- 压力
- 噪音
- 体温
- 呼吸率
- 身体电量
- HRV
- 多运动
- GPS
- 游泳
- 情绪健康
- 合并运动数据

9.11 其他模块入口

`libManager` 还挂载了许多业务模块：

- `funTable`
- `deviceInfo`
- `deviceBind`
- `syncData`

- `transFile`
- `exchangeData`
- `messageIcon`
- `deviceLog`
- `tools`
- `callNotice`
- `cache`
- `measure`

这意味着正式项目里，页面通常不需要分别 new 多个服务对象，而是从 `libManager` 统一进入。

---

10. demo 中的页面可用条件

功能页 function_index.dart 对按钮可点击状态做了明确限制。

10.1 状态变量

页面维护两个关键状态：

- `isConnected`
- `isBinded`

并根据这两个状态控制按钮是否可点。

10.2 连接相关

- 未连接时，允许“连接”
- 已连接时，允许“断开连接”

10.3 绑定相关

- 已连接且未绑定，允许“绑定”
- 已连接且已绑定，允许“解绑”

10.4 业务能力相关

以下页面入口都要求：

- 已连接
- 已绑定

包括：

- Alexa
- 同步
- 获取类功能页
- 设置类功能页

这个设计隐含表达了 demo 的接入原则：

- 蓝牙连接成功，不代表业务功能可以立刻使用
- 很多能力需要设备已经完成协议接入并具备绑定状态

---

11. 建议按什么顺序接入这两个库

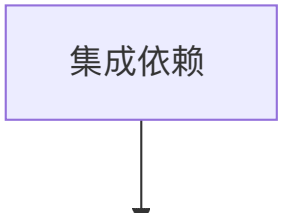
如果你要把 demo 中这套逻辑迁移到自己项目，可以按下面顺序做。

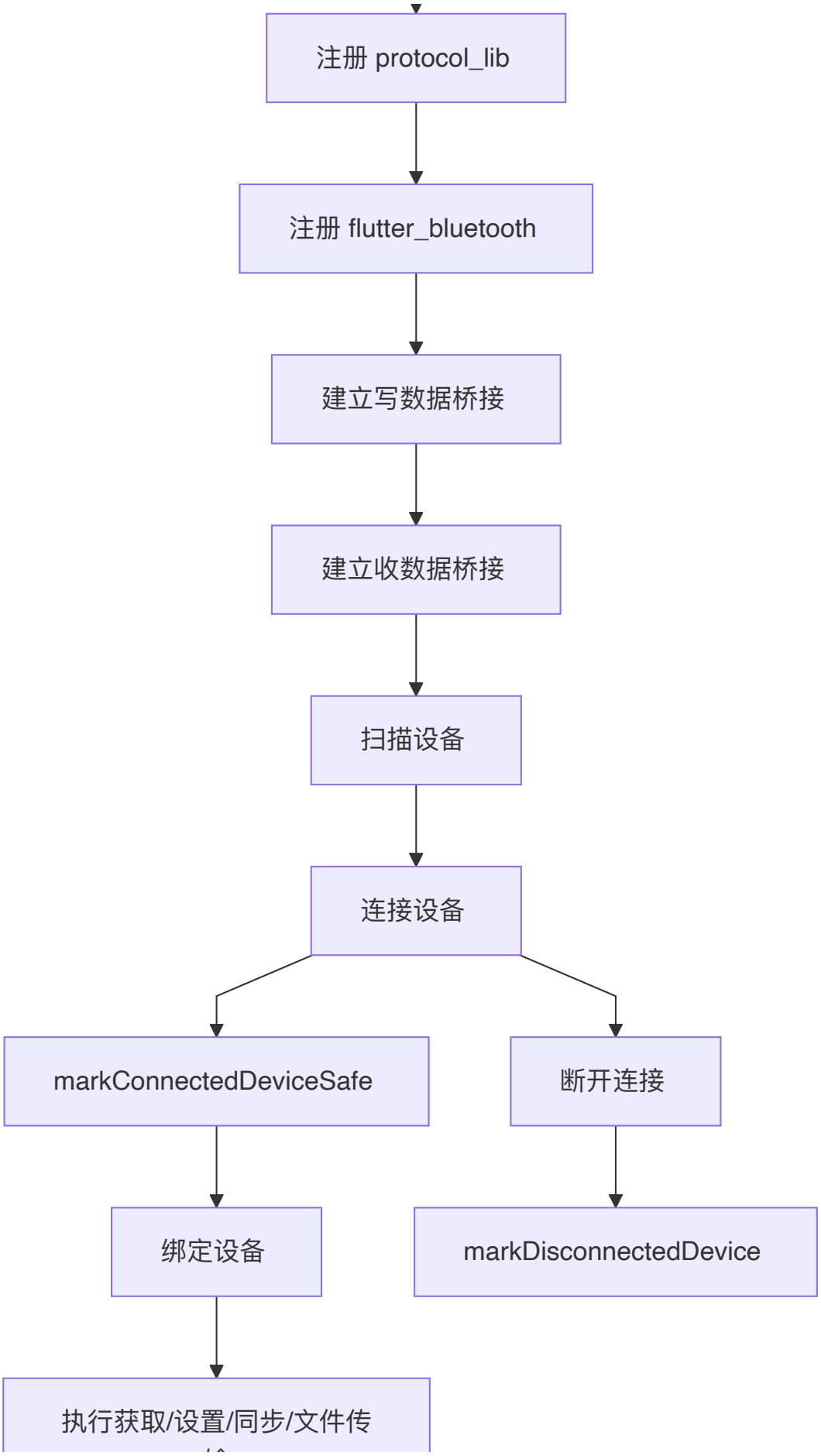
11.1 最小接入顺序

1. 集成 `protocol_lib` 与 `flutter_bluetooth`
2. App 启动时分别调用：
    - `IDOProtocolLibManager.register(...)`
    - `bluetoothManager.register()`
3. 建立桥接：
    - `receiveData()` -> `receiveDataFromBle(...)`
    - `registerWriteDataToBle(...)` -> `writeData(...)`
    - `writeState()` -> `writeDataComplete()`
4. 做蓝牙扫描与设备选择
5. 蓝牙连接成功后调用 `markConnectedDeviceSafe(...)`
6. 蓝牙断开后调用 `markDisconnectedDevice(...)`
7. 页面通过 `libManager` 调用绑定 / 查询 / 设置 / 同步等业务能力

> **Important**
> **第 5 步和第 6 步必须成对实现，不能只做其中一步。**

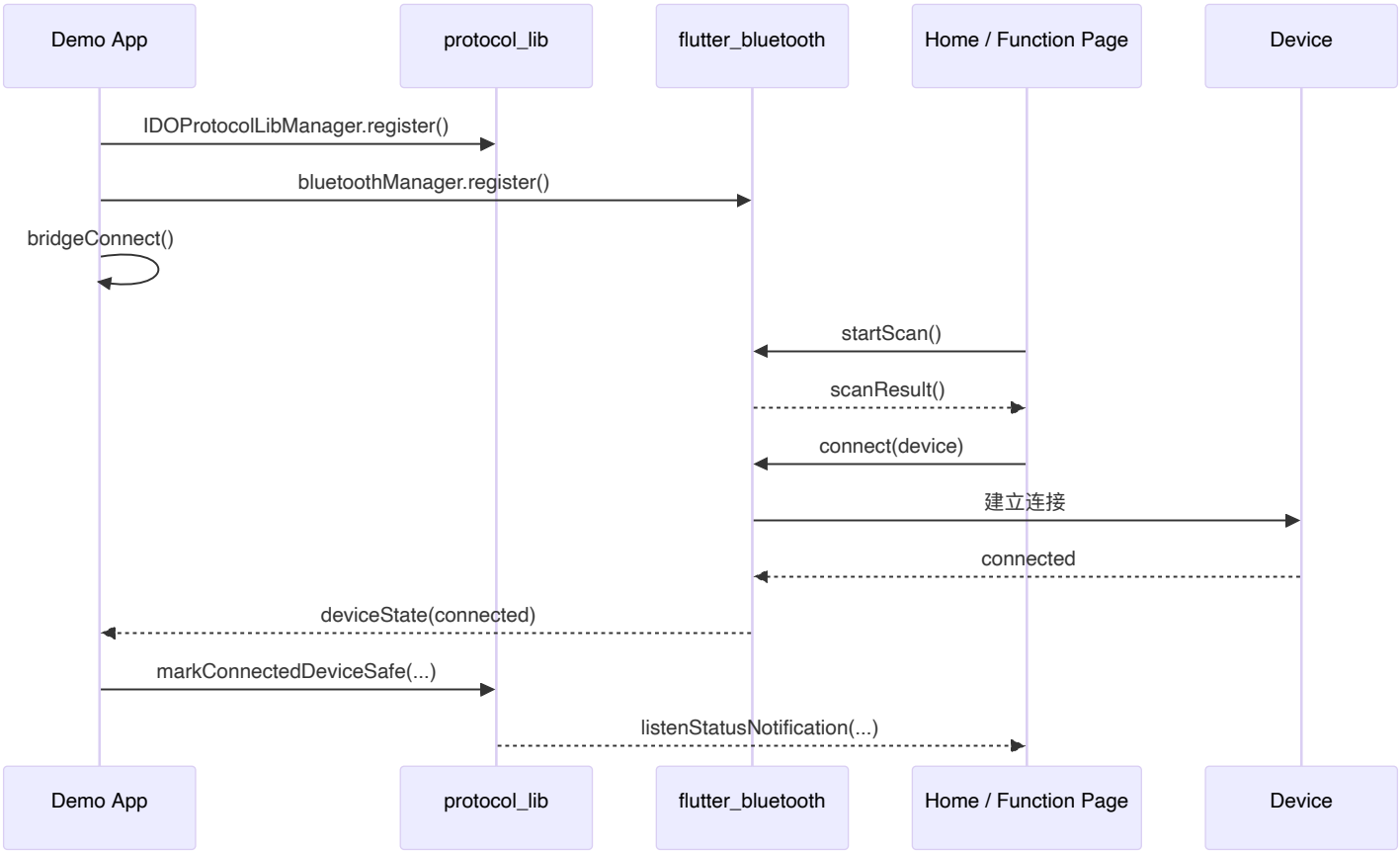
11.2 推荐接入流程图



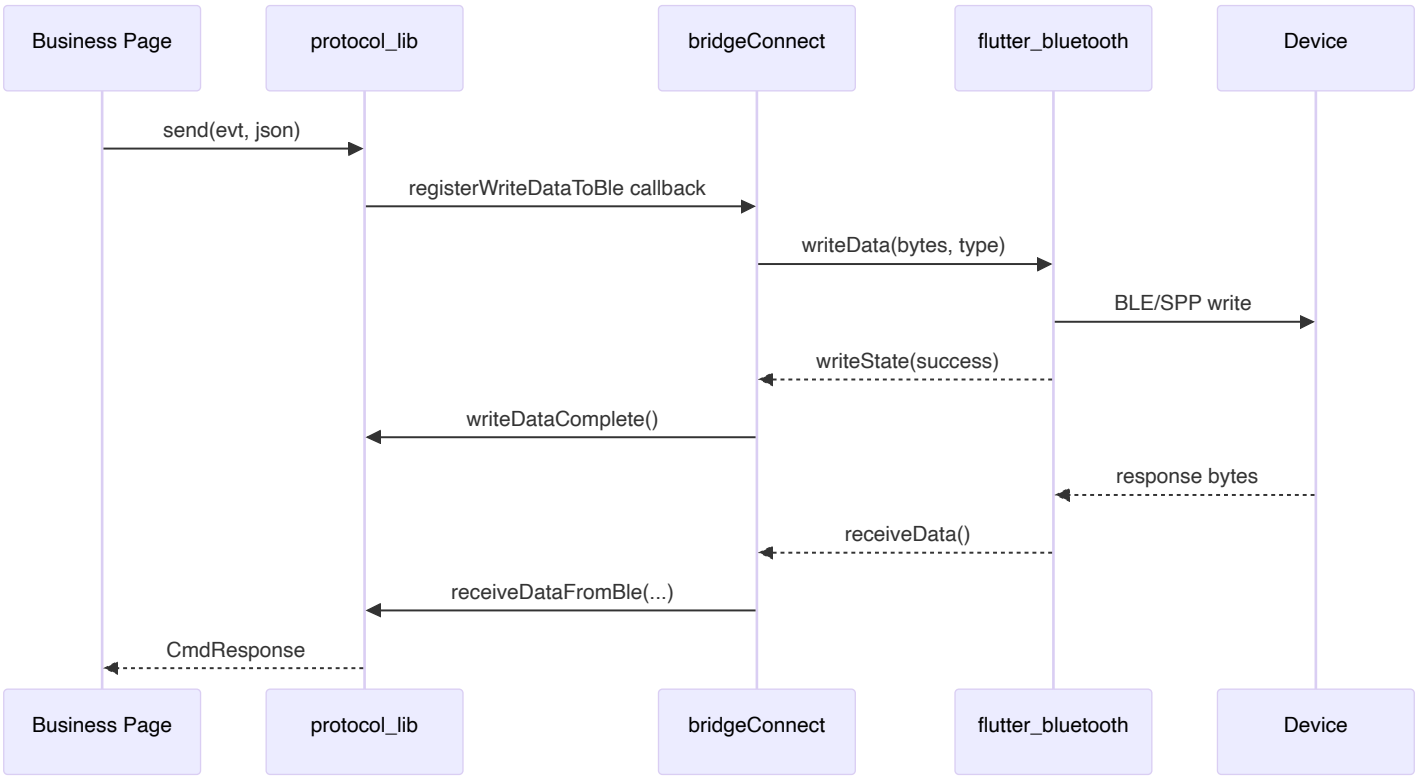

注册 protocol_lib
注册 flutter_bluetooth
建立写数据桥接
建立收数据桥接
扫描设备
连接设备
markConnectedDeviceSafe
断开连接
绑定设备
markDisconnectedDevice
执行获取/设置/同步/文件传

12. 时序图

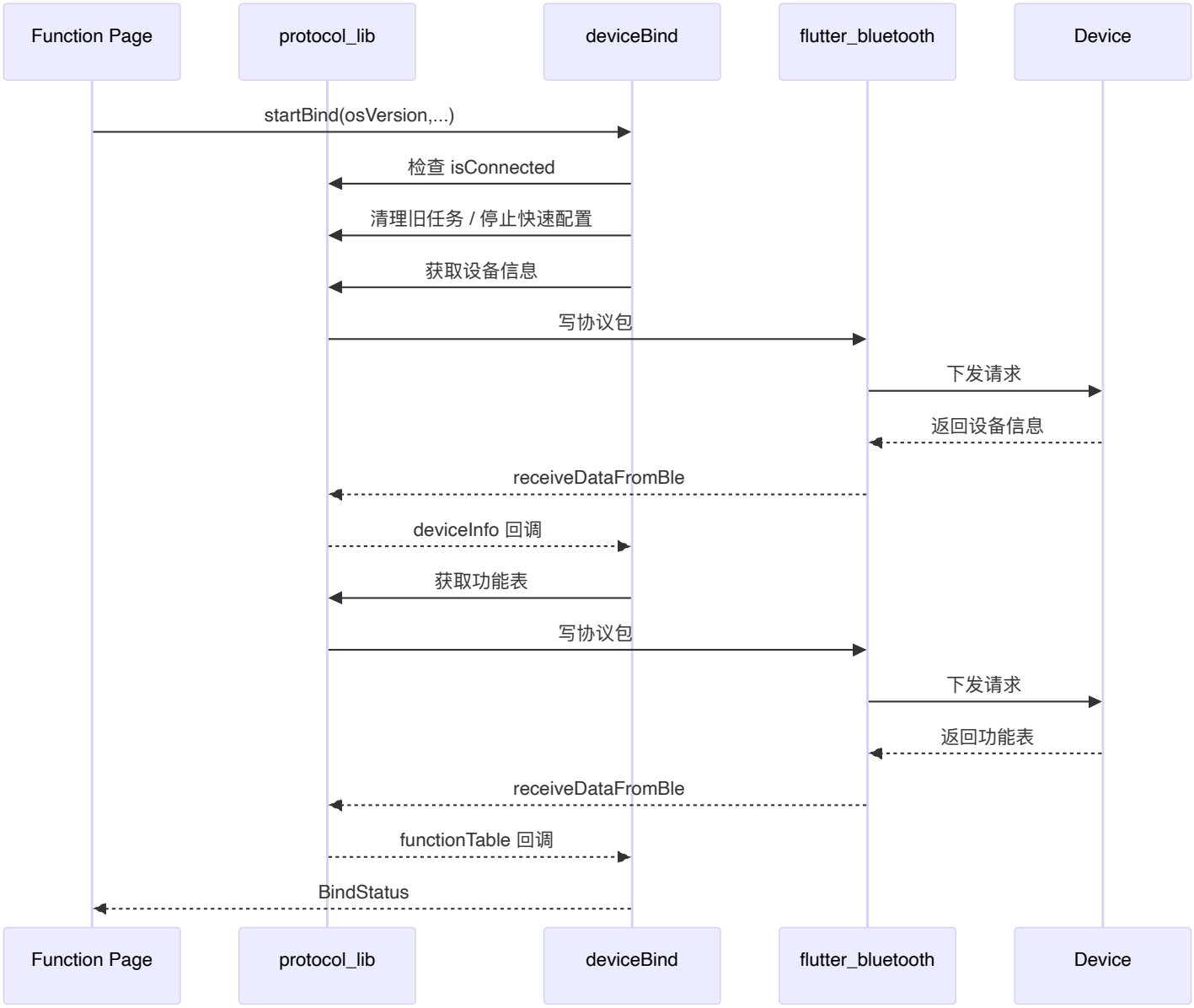
12.1 启动与接入时序图



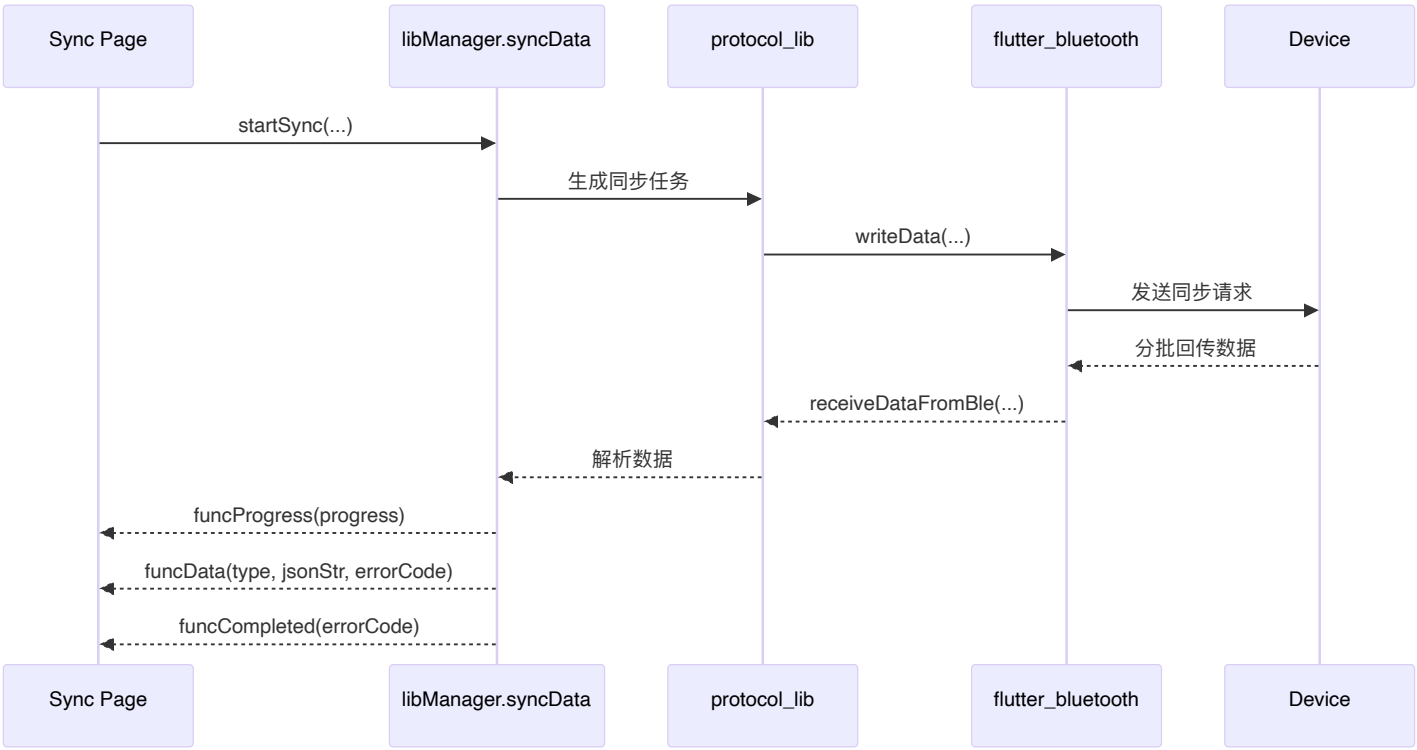
12.2 指令发送与响应时序图


Business Page
protocol_lib
bridgeConnect
flutter_bluetooth
Device
send(evt, json)
registerWriteDataToBle callback
writeData(bytes, type)
BLE/SPP write
writeState(success)
writeDataComplete()
response bytes
receiveData()
receiveDataFromBle(...)
CmdResponse
Business Page
protocol_lib
bridgeConnect
flutter_bluetooth
Device

12.3 绑定流程时序图


Function Page
protocol_lib
deviceBind
flutter_bluetooth
Device
startBind(osVersion,...)
检查 isConnected
清理旧任务 / 停止快速配置
获取设备信息
写协议包
下发请求
返回设备信息
receiveDataFromBle
deviceInfo 回调
获取功能表
写协议包
下发请求
返回功能表
receiveDataFromBle
functionTable 回调
BindStatus
Function Page
protocol_lib
deviceBind
flutter_bluetooth
Device

12.4 同步流程时序图



---

13. 关键源码位置索引

13.1 demo 入口与桥接

- main.dart

13.2 扫描与设备列表

- home_page.dart
- home_content.dart

13.3 功能页、连接、绑定、解绑

- function_index.dart

13.4 同步页

- function_sync.dart

13.5 `flutter_bluetooth` 入口

- ido_bluetooth.dart
- ido_bluetooth_manager.dart

- ido_bluetooth_manager.dart

13.6 `protocol_lib` 入口

- protocol_lib.dart
- ido_protocol_lib.dart
- ido_device_bind.dart
- device_bind_impl.dart
- ido_sync_data.dart

---

14. 总结

`demo` 的核心接入思路可以概括为一句话：

- `flutter_bluetooth` 负责连接设备和收发原始字节
- `protocol_lib` 负责协议生命周期、指令、绑定、同步与业务封装
- `main.dart` 中的桥接逻辑负责把这两者拼成完整设备通信链路

因此，真正稳定的使用方式不是“页面想发什么就直接发蓝牙包”，而是：

1. 蓝牙库建立连接
2. 协议库接管设备上下文
3. 页面调用协议业务 API
4. 协议库通过蓝牙库发送和接收底层数据

> **Important**
> **整篇文档最需要记住的一条：**
>
> - **`markConnectedDeviceSafe(...)` 与 `markDisconnectedDevice(...)` 是 `protocol_lib` 使用的关键入口**
> - **它们必须成对出现**
> - **它们分别对应协议上线与协议下线**